

Jahangirnagar University
Department of Computer Science and Engineering
Tutorial Examination, 4th Year 2nd Semester B.Sc. (Hons.) 2021-2022
CSE-451: Data Mining and Big Data Analysis Marks: 30

1. Explain 3 V's of Big Data. What is Big Data analytics?
2. Illustrate the features of NoSQL. Explain in-Database analytics.
3. What is Hadoop? Write down the features of Hadoop.
4. Describe the data processing framework of Hadoop.
5. Explain different types of NoSQL database with example.
6. What is HDFS? Explain HDFS organization and functioning.

Q1. Explain 3 V's of Big Data. What is Big Data analytics? 📖 Pages 45–46, 48

Big Data

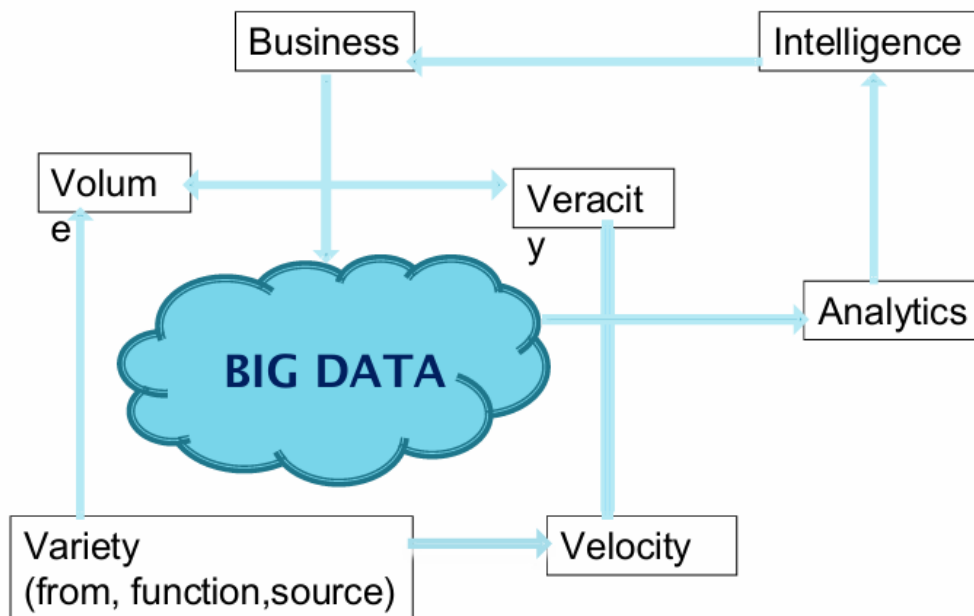
Evolution of Big Data

Definition

As in Source: Gartner IT Glossary

- Big-data is high-volume, high-velocity, and high-variety information assets that demand cost effective, innovative forms of information processing, for enhanced insight and decision making.
- Big data is data that is big in volume, velocity, and variety
- Extremely large, very fast, highly diverse, and complex data that cannot be managed with traditional data management tools. See Figure.





Big Data Analytics

Big Data Analytics

It is technology enabled analytics-

- A quite few data analytics and visualization tools are available e.g., from SAS, R Analytics to help process and analyze big data.
- About gaining a meaningful, deeper and richer insight into the business, better leveraging the services of the vendors and suppliers.
- Enables to obtain findings that allow quicker and better decision making in a competitive edge over the competitors
- A handshake between 3 communities- IT, business users and data scientists

Q2. Illustrate the features of NoSQL. Explain in-database analytics. ⚠ Not found in the

Features of NoSQL:

- 1. Schema-less / Flexible Schema** NoSQL databases do not require a fixed schema. Each document or record can have different fields, making it easy to store varied data structures. For example, in MongoDB one document can have 3 fields while another has 10.
- 2. Horizontal Scalability** NoSQL scales horizontally by adding more servers (nodes) to the cluster rather than upgrading a single powerful machine. This makes it ideal for handling massive data growth.
- 3. High Performance** NoSQL is optimized for read and write operations on large datasets. It avoids complex joins and transactions, which makes queries much faster.
- 4. Distributed Architecture** Data is distributed across multiple nodes automatically. This ensures no single point of failure and allows the system to serve millions of requests simultaneously.
- 5. Support for Unstructured/Semi-structured Data** Unlike relational databases that require structured tabular data, NoSQL can store text, JSON, XML, images, videos, and other unstructured formats.
- 6. High Availability and Fault Tolerance** NoSQL systems replicate data across multiple nodes. If one node fails, another automatically takes over, ensuring continuous availability.
- 7. BASE Properties (instead of ACID)** NoSQL follows BASE — Basically Available, Soft state, Eventually consistent — which is more suitable for distributed systems where strict consistency is relaxed in favor of availability.
- 8. Open Source and Cost Effective** Most NoSQL databases like MongoDB, Cassandra, and CouchDB are open source and run on common hardware, reducing infrastructure costs.



In-database Analytics:

In-database analytics refers to performing analytical operations directly inside the database engine rather than extracting data to an external system for processing. Key points:

- Data does not need to be moved or exported; analysis runs where data lives
 - It reduces data transfer overhead and speeds up results significantly
 - SQL-like analytical functions (aggregations, statistical operations, machine learning) are executed within the database itself
 - Tools like Oracle Advanced Analytics, IBM Netezza, and Teradata support this
 - Particularly useful for Big Data scenarios where moving large volumes of data is expensive and slow
 - NoSQL databases like MongoDB support in-database analytics through their **Aggregation Pipeline**, which processes data in stages directly within the database
-

Q3. What is Hadoop? Write down the features of Hadoop. 📄 Pages 9–11

Hadoop

Hadoop

- Hadoop is a distributed system like distributed database
- Deals with flat files in any format
- Operates on no schema for data
- Divides files automatically into blocks.

Features

- Apache Hadoop is an open-source framework designed to store and process very large datasets (Big Data) across clusters of computers. It has **several key features** that make it powerful for large-scale data processing.

1. Distributed Storage (HDFS)

- Hadoop uses **Hadoop Distributed File System** to store data.
 - Data is split into blocks and stored across multiple machines in a cluster.
 - This allows **huge datasets (terabytes or petabytes)** to be stored efficiently.
- 

2. Distributed Processing

- Hadoop processes data using **MapReduce**.
- Tasks are divided into smaller parts and processed in parallel across many nodes.
- This greatly **speeds up large data processing**.

3. Scalability

- Hadoop clusters can easily grow by **adding more machines (nodes)**.
- It can scale from a few computers to **thousands of nodes**.

4. Fault Tolerance

- Hadoop automatically **replicates data across different nodes**.
- If one machine fails, data can be recovered from another copy.

5. Cost Effective

- Runs on **commodity hardware (regular inexpensive servers)** instead of expensive supercomputers.
- The software itself is **open source**.



hadoop

6. High Throughput

- Optimized for **processing large volumes of data quickly**, rather than low-latency queries.

7. Flexibility

- Hadoop can process **structured, semi-structured, and unstructured data** (text, logs, images, etc.).

8. Data Locality

- Hadoop moves **computation to where the data is stored**, reducing network traffic and increasing performance.

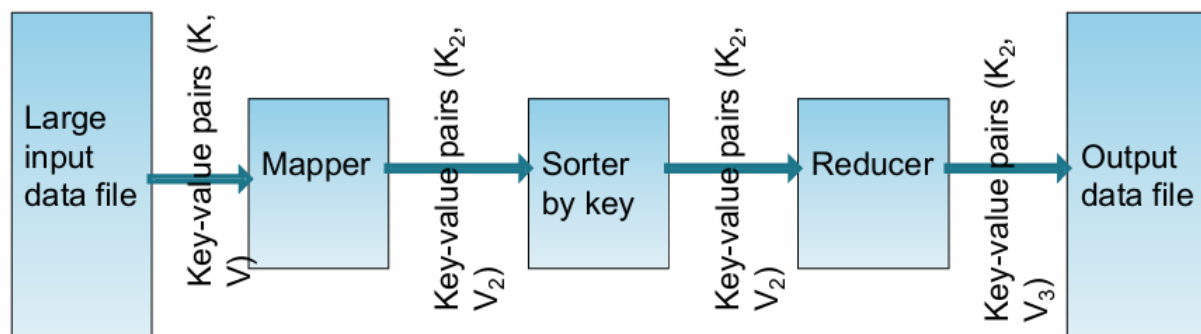
Q4. Describe the data processing framework of Hadoop. 📖 Pages 54–57

Big Data Processing

MapReduce Programming

- It is a parallel programming framework for speeding up large scale data processing for certain types of tasks
- It has minimal movement of data in distributed file systems on Hadoop clusters
- It speeds up the computation by reading and processing small chunks of file by different computers in parallel
- Map step distributes the full job into smaller tasks that can be done on separate computers using a part of the data set producing intermediate results
- Reduce step read the intermediate results, combine all of them and produce the result

MapReduce Architecture



MapReduce Architecture

- To manage the variety of data structures stored in the file system, data is stored as one key and one non-key attribute value as a key-value pair.
- Both of the input and output data must be represented in key-values formats
- Map step reads data in key-value pair format, and produces results in key-value pair format, also called key2-value2 pairs
- Reduce step reads this, and produces an output using the same keys, which can also be labeled key2-value3 format.

MapReduce Programming Components

MapReduce Programming Components

- A **Mapper** commonly performs input format parsing, projection (selection), and filtering.
- **Reducer** typically combines (adds or averages) those values.

Example.

A program for word counting of all unique words in a text.

Q5. Explain different types of NoSQL database with example. ⚠ Not found in the slides

1. Key-Value Stores

This is the simplest type. Data is stored as a collection of key-value pairs, similar to a dictionary or hash table. The key is unique and used to retrieve the value.

- **Example:** Redis, Amazon DynamoDB
- **Use case:** Session management, caching, shopping carts
- **Illustration:**

```
Key      → Value
"user:101" → "Alice"
"age:101" → "25"
```

2. Document Stores

Data is stored as documents, typically in JSON or BSON format. Each document is self-contained and can have a different structure. This is the most widely used NoSQL type.

- **Example:** MongoDB, CouchDB
- **Use case:** Content management, e-commerce, user profiles
- **Illustration:**

```
json
{
  "_id": 1,
  "name": "Alice",
  "age": 25,
  "hobbies": ["reading", "coding"]
}
```

3. Column-Family Stores (Wide-Column Stores)

Data is stored in tables but organized by columns rather than rows. Related columns are grouped into "column families." This allows very fast reads on specific columns across millions of rows.

- **Example:** Apache Cassandra, HBase
- **Use case:** Time-series data, IoT data, analytics on large datasets
- **Illustration:**

```
Row Key → Column Family: Personal    → Column Family: Contact
"Alice" → {name:"Alice", age:25}     → {email:"a@x.com", phone:"123"}
```

4. Graph Databases

Data is stored as nodes (entities) and edges (relationships). This type is ideal when the relationships between data points are as important as the data itself.

- **Example:** Neo4j, Amazon Neptune
- **Use case:** Social networks, fraud detection, recommendation engines
- **Illustration:**

```
(Alice) --[FRIENDS_WITH]--> (Bob)
(Bob)   --[WORKS_AT]-----> (Google)
```

Q6. What is HDFS? Explain HDFS organization and functioning. 📄 Pages 7, 18–20

1. HDFS:

- i. Storage components
- ii. Distributes data across several nodes
- iii. Natively redundant

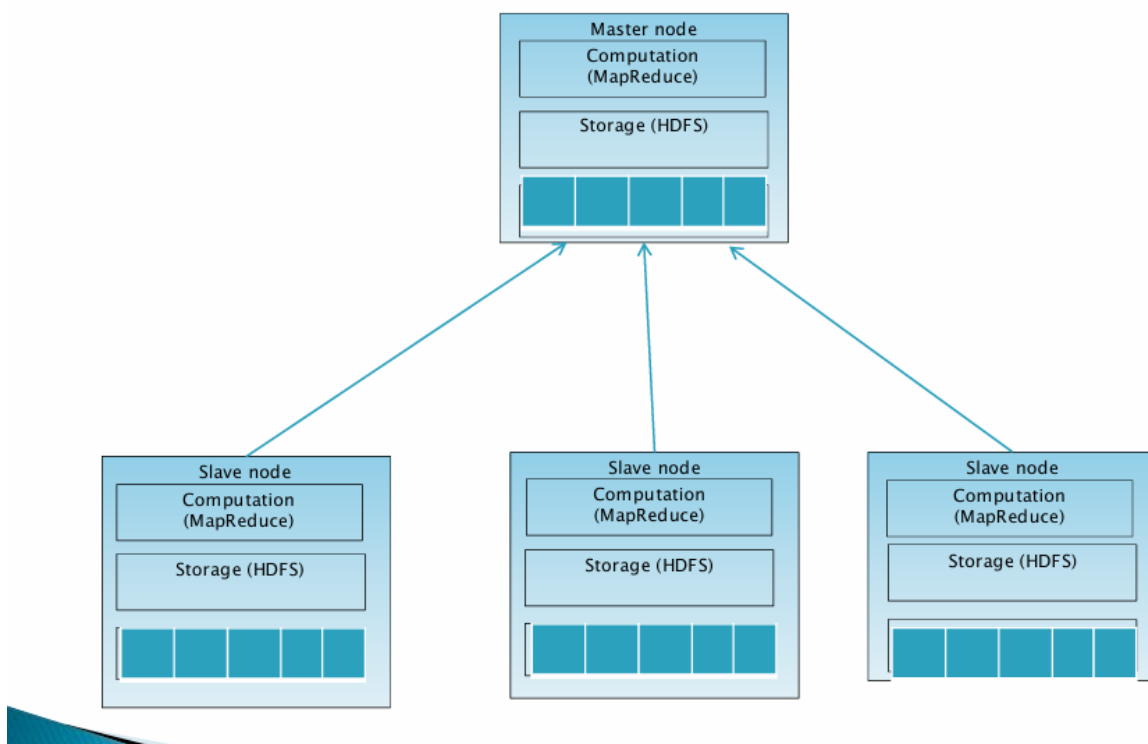
High level architecture of Hadoop

- Hadoop is a distributed Master-Slave architecture
- Master node is known as Namenode, and slave nodes are known as Datanodes

Key components of the Master node

1. Master HDFS: : Partitioning the data storage across the slave nodes.
Keeps track of locations of data on DataNodes.
1. Master MapReduce: It decides and schedules computation tasks on slave nodes.

High level Architecture of Hadoop



Functioning: HDFS splits files into blocks and distributes/duplicates them across several nodes. Local compute resources process each chunk in parallel. The framework handles failover automatically, ensuring fault tolerance. Key HDFS commands include listing files (`hdfs dfs -ls`), uploading (`-put`), downloading (`-get`), copying (`-cp`), and moving (`-mv`) files.